

Lecture 10: Outline

- ▶ The gradient algorithm:
 - ▶ formalization,
 - ▶ discussion of convergence properties,
 - ▶ discussion of parameters,
 - ▶ discussion computational costs.

Lecture 10: The gradient algorithm

► Input:

- a known function $\Phi : \mathbb{R}^N \rightarrow \mathbb{R}$
- an initial parameter $\mathbf{s}^0 \in \mathbb{R}^N$.
- a given step-length multiplier/learning rate: $\alpha \in \mathbb{R}$.

► Steps:

- $\mathbf{s}^{n+1} = \mathbf{s}^n - \alpha \nabla \Phi(\mathbf{s}^n)$

► Stopping:

- One can fix the number of iterations.
- One can apply some stopping criteria (mind the speed):
 - $|\nabla \Phi(\mathbf{s}^N)| < \varepsilon$ - at a very flat region, we stop,
 - $|\mathbf{s}^{N+1} - \mathbf{s}^N| < \delta$ - for very small differences, we stop.

Lecture 10: The gradient algorithm: convergence rate

- ▷ We need: Lipschitz constant L for the gradient

- ▶ the smallest number L satisfying

$$|\nabla\Phi(\mathbf{s}_j) - \nabla\Phi(\mathbf{s}_k)| \leq L|\mathbf{s}_j - \mathbf{s}_k| \text{ for all } \mathbf{s}_j, \mathbf{s}_k. \quad (L)$$

- ▷ If Φ is convex and there exists L above, then for any learning rate $\alpha \leq \frac{1}{L}$:

$$\Phi(\mathbf{s}^n) - \min \Phi \leq 2L \cdot \frac{|\mathbf{s}^0 - \mathbf{s}^{\text{opt}}|^2}{n}$$

- ▶ Here $\Phi(\mathbf{s}^{\text{opt}}) = \min \Phi$.
- ▶ Here convexity is a strict assumption.
- ▶ Meaning: deviation from the minimum (left hand side) is decreasing with iteration number/ computational time,
 - ▶ proportional to $\frac{1}{n}$ or $\frac{1}{\text{computing time}}$.

Lecture 10: The gradient algorithm: on the assumptions

- ▶ There is no warranty for Φ being convex.
 - ▶ The algorithm can be applied also to any function but no theory for convergence.
- ▶ Would be sufficient convexity in a neighborhood V of the minimum site and launch the process from V .
- ▶ Implicitly, we need Φ to be differentiable.
 - ▶ This is problematic, the automatic differentiation sometimes, also fails to work.
 - ▶ By presence of maximization steps.

Lecture 10: The gradient algorithm: choice of α

- ▶ Mentioned in the error analysis: $\alpha \leq \frac{1}{L}$.
- ▶ Usually we have no easy estimate for L ;
 - ▶ With the second derivative we could estimate.
 - ▶ Usually: pick a “sufficiently small” value for α (e.g., below 0.3)
 - ▶ If (L) fails, choose a smaller one.
- ▶ Possible problems by choosing α - we will see in experiments:
 - ▶ Too large α : no convergence, (too long steps, we may just skip the optimal value).
 - ▶ Too small α : very slow convergence.

Lecture 10: The gradient algorithm: choice of α (continued)

- ▶ Similar small values α for very different data?
- ▶ A cautionary example.
 - ▶ Change $\Phi_* = 10\Phi$,
 - ▶ We get the same data in another unit.
 - ▶ The same minimum site.
 - ▶ Original step - computing the minimum of Φ :
$$\mathbf{s}^{n+1} = \mathbf{s}^n - \alpha \nabla \Phi(\mathbf{s}^n)$$
 - ▶ New step - computing the minimum of Φ_* :
$$\mathbf{s}^{n+1} = \mathbf{s}^n - \alpha \nabla \Phi_*(\mathbf{s}^n) = \mathbf{s}^n - 10 \cdot \alpha \nabla \Phi(\mathbf{s}^n)$$
- ▶ Ten times longer step for the same task.
 - ▶ One should normalize data!

Lecture 10: The gradient algorithm: an example I.

- ▶ Find (x, y) such that $\Phi(x, y) = 2x^2 - 4xy + y^4$ is minimal.
- ▶ Now $\nabla\Phi(\mathbf{s}) = (4x - 4y, -4x + 4y^3)$.
 - ▶ We choose $\alpha = 0.2$.
 - ▶ Start first from $\mathbf{s}^0 = (0, 0)$.
- ▶ Bad choice: $\nabla\Phi(\mathbf{s}^0) = (0, 0) \Rightarrow \mathbf{s}^0 = \mathbf{s}^1 = \mathbf{s}^2 = \dots$,
but it is not the minimum: $-\frac{7}{16} = \Phi(\frac{1}{2}, \frac{1}{2}) < f(0, 0) = 0$.
- ▶ Another choice: $\nabla\Phi(\mathbf{s}^0) = (0.1, 0.1)$.
 - ▶ A random choice is also common for this reason.
- ▶ Result of a computer code:
 $\mathbf{s}^8 = [0.873, 0.945], \mathbf{s}^{13} = [1.001, 0.982], \mathbf{s}^{19} = [1.141, 0.520]$
 - ▶ This was skipping the minimum site $(1, 1)$.

Lecture 10: The gradient algorithm: an example II.

- ▶ OK, change the learning rate:

- ▶ We choose $\alpha = 0.1$

- ▶ Start again from $\mathbf{s}^0 = (0.1, 0.1)$.

- ▶ Result of a computer code:

$$\mathbf{s}^8 = [0.386, 0.567], \mathbf{s}^{13} = [0.742, 0.874], \mathbf{s}^{19} = [0.941, 0.975]$$

- ▶ Convergence (even for such an easy problem) is slow.

- ▶ The entire method could be developed.

- ▶ Another problem:

- ▶ Start now from $\mathbf{s}^0 = (-0.1, -0.1)$ with $\alpha = 0.1$.

- ▶ Result of a computer code:

$$\mathbf{s}^8 = [-0.39, -0.57], \mathbf{s}^{13} = [-0.74, -0.87], \mathbf{s}^{19} = [-0.94, -0.97]$$

- ▶ Even the opposite: which is the correct one?

Lecture 10: setting parameters I.

- ▶ Choice of initial $\mathbf{s}^0$: no general rule.
 - ▶ If parameters of the network have a real meaning, we can have some guess.
- ▶ Choice of the learning rate: Try to make it adaptive.
- ▶ Idea I: apply more analysis
 - ▶ Approximations to find optimal $\alpha_{n,\text{opt}}$ with
$$|\mathbf{s}^{\text{opt}} - (\mathbf{s}^n - \alpha_{n,\text{opt}} \nabla \Phi(\mathbf{s}^n))| \rightarrow \min$$
 - ▶ This requires some analysis, quadratic approximation, where $\nabla^2 \Phi(\mathbf{s}^n)$ has to be computed.
 - ▶ This is too much work.

Lecture 10: setting parameters II.

- ▶ Idea II: apply the formula

$$\alpha_k = \left| \frac{(\nabla\Phi(\mathbf{s}^k) - \nabla\Phi(\mathbf{s}^{k-1})) \cdot (\mathbf{s}^k - \mathbf{s}^{k-1})}{|\nabla\Phi(\mathbf{s}^k) - \nabla\Phi(\mathbf{s}^{k-1})|^2} \right|$$

- ▶ Realistic: larger steps in the “flat” regions.
 - ▶ No extra effort: the quantities here were already computed.
 - ▶ Improved convergence analysis with further assumptions.
- ▶ Idea III: apply the formula

$$\alpha_n = \frac{\alpha_0}{n+1} \quad \text{or} \quad \alpha_n = \frac{\alpha_0}{\sqrt{n+1}}.$$

- ▶ This is somewhat heuristic but prevents too long approximations.

Lecture 10: Outline

- ▶ Extensions of the gradient (descent) method
 - ▶ conjugate gradient method,
 - ▶ summarizing computational costs,
 - ▶ stochastic gradient method.

Lecture 10: How to develop /change the gradient method?

- ▶ We have realized:
 - ▶ Main problem in gradient descent: find appropriate **directions** and step-sizes in the iterations.
 - ▶ Yes, directions should also be changed to seek the minimum in other directions. Maybe by exploiting the geometry of the problem.
- ▶ The general shape: $\mathbf{s}^{n+1} = \mathbf{s}^n - \alpha_n \mathbf{v}^n$, where
 - ▶ $\mathbf{v}^n$ is the search direction
 - ▶ Previously: $\mathbf{v}^n = \nabla \Phi(\mathbf{s}^n)$.
 - ▶ Next method: appropriate choice of $\mathbf{v}^n$.

Lecture 10: Choosing optimal directions in optimization

- ▶ A theoretical result for strictly convex quadratic functions Φ :
 - ▶ Optimal directions are conjugate with respect to the matrix $\nabla^2\Phi$.
 - ▶ Formally: for $n \neq k$: $\mathbf{v}^n \cdot \nabla^2\Phi \mathbf{v}^k = 0$.
 - ▶ Therefore, its name: conjugate gradient method.
- ▶ In this “raw” form: we had to compute $\nabla^2\Phi$: too much computation.
- ▶ Instead, the optimal direction is found also by an iterative process.

Lecture 10: Choosing optimal directions: the Fletcher–Reeves method

- ▶ Start with a (good) $\mathbf{s}^0$ and initial direction $\mathbf{d}_0 = \nabla\phi(\mathbf{s}^0)$.
- ▶ For $k = 0, 1, \dots, n - 1$
 - ▶ Minimize in direction $\mathbf{d}_k$:

Choose $\alpha_k \in \mathbb{R}$: $\Phi(\mathbf{s}^k - \alpha_k \mathbf{d}_k)$ is minimal.

$$\mathbf{s}^{k+1} = \mathbf{s}^k - \alpha_k \mathbf{d}_k,$$

- ▶ $\mathbf{g}_{k+1} = \nabla\Phi(\mathbf{s}^{k+1})$ - candidate for the new direction.
- ▶ If $k < n - 2$, correct it to compute the new direction:

$$\mathbf{d}_{k+1} = \mathbf{g}_{k+1} + \frac{\|\mathbf{g}_{k+1}\|^2}{\|\mathbf{g}_k\|^2} \mathbf{d}_k.$$

- ▶ We finally have $\mathbf{g}_n$ and $\mathbf{s}^n$.
- ▶ Go back to the first step: if $\nabla\Phi(\mathbf{s}^n)$ or $\Phi(\mathbf{s}^n)$ is not acceptable, proceed again with $\mathbf{s}^0 = \mathbf{s}^n$.

Lecture 10: Summarizing computational costs I.

- ▶ Based on the principle to find an appropriate multi-dimensional surface determined by $\mathbf{a} \in \mathbb{R}^M$, we have to minimize:

$$\sum_{k=1}^K \psi(\Phi_{\mathbf{a}}(\mathbf{x}_k) - \mathbf{y}_k)$$

- ▶ $\mathbf{x}_k$: input data (of number k)
- ▶ $\mathbf{y}_k$: output data (of number k) .
- ▶ ψ loss function.

In each step, we have to compute the gradient:

$$\left(\sum_{k=1}^K \psi(\Phi_{\mathbf{a},b}(\mathbf{x}_k) - \mathbf{y}_k) \right)' = \sum_{k=1}^K (\psi(\Phi_{\mathbf{a},b}(\mathbf{x}_k) - \mathbf{y}_k))' \quad (1)$$

Lecture 10: Summarizing computational costs II.

- ▶ Computation of one gradient (one term in the sum):

- ▶ Now with $\Phi = f_N \circ f_{N-1} \circ \dots \circ f_1$

$$\psi'(\mathbf{f}_N \circ \dots \circ \mathbf{f}_2 \circ \mathbf{f}_1(\mathbf{x})) \cdot \mathbf{f}'_N(\mathbf{f}_{N-1} \circ \dots \circ \mathbf{f}_2 \circ \mathbf{f}_1(\mathbf{x})) \cdot \dots \cdot \mathbf{f}'_1(\mathbf{x})$$

- ▶ For this: matrix-vector multiplications with a number of

$$(d_N \cdot d_{N-1}) \cdot ((d_{N-1} \cdot d_{N-2})) \cdot \dots \cdot ((d_1 \cdot d_0))$$

multiplications.

- ▶ This can be rather large.
- ▶ Alternative: We (indeed, the computer) should compute the gradient in symbolic way as a function.
- ▶ In that case: evaluate the gradient vector of size M .

Lecture 10: Summarizing computational costs III.

- ▶ Number of operations until now depend roughly on the number of parameters.
- ▶ We can hardly save on this:
 - ▶ by fixing the number of parameters (or $f_1, f_2, \dots, f_N$ given by the layers) the gradient should be computed.
- ▶ Do not forget; we should perform it K -times.
 - ▶ For 10^5 input-output pairs, this can be long.
 - ▶ Usually: a number of similar input-output pairs.

Lecture 10: Preparation of stochastic gradient method

- ▶ Instead of taking an “entire sum” in (1):
 - ▶ Take $K_b \ll K$ and consider the input-output pairs with $\{1, 2, \dots, K_b\}$.
 - ▶ Accordingly, in the sum (1) take $k = 1, 2, \dots, K_b$.
 - ▶ Take a gradient-step based on the above sum.
 - ▶ Repeat it for $k = K_b + 1, K_b + 2, \dots, 2K_b$ and further until all input-output pairs are used.
- ▶ This method is still deterministic.
- ▶ Usually, this results a better approximation compared to a single conventional gradient step.
- ▶ This is OK, but using $1, 2, \dots, K_b$ is arbitrary.

Lecture 10: Principle of stochastic gradient method

- ▶ Slightly develop the above algorithm:
 - ▶ Take a *random sample* of number K_b from $\{1, 2, \dots, K\}$ and consider the input-output pairs with these indices.
 - ▶ In the sum (1) use only these indices.
 - ▶ Take a gradient-step based on the above sum.
 - ▶ Repeat it as long as necessary.
- ▶ It is also applied for $K_b = 1$.
- ▶ It pays off, if K is large.
- ▶ Usually does not deliver a very precise minimum by itself. Reducing the learning rate over time or increasing K_b can improve this.
- ▶ For small K_b : relative large oscillations in the final loss.